

Phase 1

PDF Ingestion Pipeline

Ollama · Embeddings · PyMuPDF · Chunking · Qdrant Client · Idempotent Upserts · FastAPI File Uploads

Today's win in one line: Uploading a PDF to `POST /ingest/pdf` now runs a real end-to-end pipeline — text extraction, chunking, embedding, and vector storage with page-number metadata — and returns a JSON report. Ten chunks of your Day 1 study guide are permanently living in Qdrant as 768-dimensional semantic fingerprints.

What you built / verified	How
Ollama running locally	Installed, service on <code>localhost:11434</code> , verified with <code>curl</code> .
<code>nomic-embed-text</code> embedding model pulled	~275 MB pull; 768-dim vector verified end-to-end.
Central config (<code>.env</code> + <code>config.py</code>)	Single source of truth for URLs, model, dimension, chunk sizes.
PDF parser (<code>parsers/pdf_parser.py</code>)	PyMuPDF; bytes → list of (page_number, text).
V1 chunker (<code>chunking/simple_chunker.py</code>)	Sliding-window with overlap; ~700-token chunks, ~100-token overlap.
Embedding client (<code>embeddings/embedding_client.py</code>)	httpx wrapper around Ollama, with dimension-pin enforcement.
Qdrant store (<code>vector_store/qdrant_store.py</code>)	Client wrapper, deterministic UUIDs, upsert + delete + count.
Ingestion service (<code>services/ingestion_service.py</code>)	Orchestrates the pipeline; returns an <code>IngestionReport</code> .
<code>POST /ingest/pdf</code> endpoint	Multipart upload; validation; 201 Created; JSON report.
Idempotency verified	Re-upserting the same file doesn't duplicate points.
Semantic similarity verified	"revenue Q2" vs "income last quarter" = 0.72; vs "weather" = 0.45.

Companion docs: AI Ingestion Engine Build Plan · AI Ingestion Engine Decisions · Day 1 Study Guide

1 Ollama and embedding models

1.1 What Ollama is (in your voice)

DEFINITION A program that runs AI models locally on your machine and exposes them as an HTTP API at `localhost:11434`. Same shape as any web service — POST text, get JSON back.

WHERE USED Local development, offline inference, cost-sensitive prototypes, privacy-sensitive apps that can't send data to cloud LLM APIs.

EXAMPLE You `ollama pull nomic-embed-text` once (like `docker pull`), then POST to `/api/embeddings` with a model name and a prompt. Ollama returns a JSON with an `embedding` field: 768 floats.

KEY TERMS **Model** = the AI file. **Pull** = download. **Serve** = expose as API. **Prompt** = the input text. Ollama Cloud is the same API hosted remotely — swap the base URL when your laptop is too weak.

WHY FOR THE INTERNSHIP Your instructor's brief specifically mentioned Ollama cloud models. Using local Ollama for embeddings + being able to switch to cloud with one env var change is the flexibility the plan called for.

1.2 What an embedding actually is

An embedding is a list of numbers — a **vector** — that represents the *meaning* of a piece of text. The model learned during training that similar meanings should produce similar vectors.

- "revenue increased in Q2" → 768 floats pointing in some direction in 768-dim space.
- "income went up last quarter" → 768 floats pointing in *almost the same direction*.
- "the weather in Beirut is warm" → 768 floats pointing somewhere else entirely.

You can then compare vectors mathematically (cosine similarity) to answer **"how similar in meaning are these two texts?"** — even when they share zero exact words. This is what makes RAG search work.

YOU PROVED THIS TODAY

In Test 3 of the embedding client, `similarity("revenue Q2", "income last quarter") = 0.72` and `similarity("revenue Q2", "weather Beirut") = 0.45`. The model rated the first pair much more similar despite them sharing no words. That gap is the entire reason semantic search beats keyword search.

1.3 Why `nomic-embed-text` specifically

- **Retrieval-tuned** — trained specifically for "how similar are these two texts" (not chat, not classification). That's exactly the operation RAG needs.
- **Free and open-source** — no API keys, no per-call cost.
- **Small enough for CPU** — 274 MB, runs comfortably even without a GPU. Matches your hardware.
- **768 dimensions** — matches the Qdrant collection you pinned in Phase 0. This is Decision 2 becoming a physical, verified fact.

2 PDF parsing with PyMuPDF

2.1 What PyMuPDF does and why it's the right pick

DEFINITION A Python binding for MuPDF, a fast open-source PDF renderer/parser written in C. Import name is confusingly `fitz` (the original name of the engine).

WHERE USED Anywhere you need to extract text, images, or render pages from PDFs. Preferred over pdfplumber/pdfminer for speed and reliability on real-world PDFs.

EXAMPLE

```
doc = fitz.open(stream=pdf_bytes, filetype="pdf"); for page in doc: text = page.get_text()
```

KEY TERMS **doc** = open PDF document. **page** = one page object. **get_text()** = plain text extraction. **stream** = read from memory instead of disk (important for uploaded files).

WHY FOR THE INTERNSHIP The build plan calls PyMuPDF the "first choice" for the PDF parser stage of your ingestion engine. It handles most real PDFs cleanly and is fast enough for interactive uploads.

2.2 What your parser does — and doesn't

Your `extract_pdf_pages` function is intentionally minimal: bytes in, list of `(page_number, page_text)` out. Single responsibility.

- Uses 1-based page numbers because humans count pages from 1, not 0.
- Returns empty text for scanned/image pages — **doesn't try to OCR them**. That's Phase 4's job.
- Doesn't clean the text — that's the chunker's/embedder's job to be robust to.
- Doesn't chunk. Doesn't embed. Doesn't touch Qdrant.

WHY "SINGLE RESPONSIBILITY" MATTERS

When you add video ingestion in Phase 5, you'll write a *different* extractor (OpenCV frames + OCR) with the same output shape: `list[(source_id, text)]`. Everything downstream reuses the exact same code. That's the payoff for keeping this module narrow.

3 Chunking

3.1 Why chunking exists at all

Two reasons, both worth being able to say out loud:

1. **Embedding models have a max input length.** You can't feed a whole document into one call.
2. **Retrieval finds one thing.** If you embedded an entire book as one vector, the vector would be a blurry average of every idea in it — searching for "revenue in Q2" would match almost anything. Small chunks = one idea per chunk = precise retrieval.

The sweet spot is a chunk big enough to hold context, small enough to represent one idea. Empirically that's ~500-900 tokens.

3.2 Why overlap matters

If chunks are clean-cut with no overlap, an important sentence sitting on a boundary gets split in half — neither chunk contains the full idea, so neither one matches a query about it well.

Overlap of ~80-150 tokens (roughly 100 for you) means the end of chunk N is repeated at the start of chunk N+1. A query near the boundary now matches at least one of them clearly.

YOU PROVED THIS TODAY

In your chunker test, the last 60 chars of chunk 0 and the first 60 chars of chunk 1 shared the same phrase "over the lazy dog. The quick brown fox jumps" — that's overlap doing its job.

3.3 Start dumb, get smart (Decision 5 in action)

Your V1 chunker splits by **character count**, using the "1 token $\approx$ 4 characters" rule of thumb (English). It doesn't respect sentences, headings, or paragraphs — that's V2. It doesn't do semantic splitting — that's V3.

The reason for shipping V1 first: **you can't tune what doesn't run.** Build the whole pipeline dumb first, verify end-to-end retrieval quality, *then* upgrade the chunker if retrieval is weak. Starting with a semantic chunker means you spend days tuning it before you have any signal on whether the rest of the pipeline works.

4 Qdrant client, points, and idempotency

4.1 The three parts of a Qdrant point

- **id** — unique identifier for the point (int or UUID).
- **vector** — the 768 floats.
- **payload** — arbitrary JSON metadata attached to the point (file_id, file_name, page_number, chunk_index, and importantly, the original text).

Why store the original text in the payload? Because when you retrieve chunks in Phase 2 (search) and Phase 3 (ask), you need the actual text to build the LLM prompt. Vectors alone are unreadable numbers.

4.2 Deterministic UUIDs = idempotent ingestion

`make_point_id(file_id, chunk_index)` uses `uuid5`, which produces the same UUID every time for the same input. So chunk 3 of `file_abc` is always the same point ID, run after run.

Consequence: **re-uploading the same file doesn't duplicate points**. Qdrant treats the second upsert as an update to the existing points. This is what you verified in Test 6 of the store — count stayed at 3 after re-upsert.

WHY THIS MATTERS

Without deterministic IDs, every retry, every accidental double-click, every dev re-run would create duplicate chunks — which would show up as duplicate results in search and slowly corrupt retrieval quality. Idempotency is a property most junior projects skip and later regret.

4.3 The dimension pin, enforced three times

Your app checks `EMBEDDING_DIM = 768` in three separate places:

1. **Config load** — `config.py` reads `EMBEDDING_DIM` and casts to int. If missing → `RuntimeError` at startup.
2. **Embedding client** — after Ollama responds, if the vector isn't exactly 768 floats → `RuntimeError` with a message pointing at the model mismatch.
3. **Qdrant store** — `ensure_collection` checks the collection's actual dim matches settings. If someone changed `.env` but not the collection → `RuntimeError`.

Three checks, one number, zero surprises. This is Decision 2 (the biggest footgun in the whole plan) turned into runtime safety.

5 FastAPI patterns you used today

5.1 Routers and `include_router`

Instead of piling every route into `main.py`, you split them into files under `app/api/`. Each file defines an `APIRouter` with its own prefix and tags. `main.py` just includes them.

```
# app/api/routes_ingest.py
router = APIRouter(prefix="/ingest", tags=["ingest"])

@router.post("/pdf")
async def ingest_pdf_endpoint(file: UploadFile = File(...)):
    ...

# app/main.py
from app.api.routes_ingest import router as ingest_router
app.include_router(ingest_router)
```

When you add `routes_search.py`, `routes_ask.py`, and `routes_ingest.py`'s video endpoint, they follow the same pattern. `main.py` stays small forever.

5.2 File uploads with `UploadFile`

`UploadFile` is FastAPI's type for multipart file uploads. It exposes:

- `file.filename` — client-provided filename (validate it).
- `file.content_type` — MIME type from the client (validate it, but don't fully trust it).
- `await file.read()` — read all bytes into memory. Async because it might stream.

Requires the `python-multipart` package installed — without it, FastAPI can't parse multipart bodies at all. This is why we installed it in Phase 1's dependency list.

5.3 Correct HTTP status codes

- **201 Created** — the response you get after a successful ingest. Not 200 (which is generic "OK"). 201 semantically means "we made a new resource." Precision matters when other services (like n8n) branch on status codes.
- **400 Bad Request** — client sent us something invalid (wrong content-type, missing extension, empty file).
- **413 Payload Too Large** — the file exceeded your `MAX_PDF_MB` limit. This is Decision 7 in code.
- **500 Internal Server Error** — default for unhandled exceptions. The pipeline can raise these; we don't try to catch every one in the MVP.

5.4 The services / routes separation

Business logic (`ingest_pdf`) lives in `services/` . HTTP glue (`ingest_pdf_endpoint`) lives in `api/` . The route validates the request, extracts bytes, calls the service, returns the report.

Why this matters: the service can be called from anywhere — a CLI script, a test, a background job — without needing to fake an HTTP request. That flexibility is what makes the code testable and portable.

YOU PROVED THIS TODAY

Your `test_ingest.py` called `ingest_pdf(pdf_bytes, file_name)` directly from a script, no HTTP involved, and it worked end-to-end. Same function the API route calls. That's the separation paying off.

6 The "one engine, two extractors" pattern

The insight your build plan was built around

The whole pipeline is six stages:

```
SOURCE → [1] EXTRACT → [2] CLEAN → [3] CHUNK → [4] EMBED → [5] STORE → [6] SEARCH/ASK
```

Only stage 1 differs by source type. PDF uses PyMuPDF; video will use OpenCV frames + OCR. Stages 2-5 are shared. In your code today, that shared tail is `_ingest_texts` inside `ingestion_service.py`. It takes chunks + metadata, calls the embedding client, calls the Qdrant store. When you write the video service in Phase 5, it will call the same `_ingest_texts` — different metadata (timestamps instead of page numbers), same helper.

This is the architectural choice that makes video feel like an add-on instead of a second project. It's why your build plan puts PDF first: to prove the shared engine on the easiest input.

Q&A Likely interview questions on today's work

Answers in your voice — the way you'd actually explain it, not a textbook.

Question	Your answer
What is an embedding, in one sentence?	A list of numbers that represents the meaning of a piece of text, so two things with similar meaning end up with similar lists — even if they don't share any words.
Why 768 dimensions specifically?	Because that's what <code>nomic-embed-text</code> outputs. The number itself isn't special — but once I picked the model, the number was fixed and my Qdrant collection had to be created with the exact same size. If they don't match, nothing works.
What does Ollama do?	It runs AI models on my machine and exposes them as a local HTTP API. My Python code talks to it the same way a browser talks to a website — POST some text, get JSON back. I use it for embeddings, and later for the answer-generation LLM in Phase 3.
Why did you pick PyMuPDF over other PDF libraries?	It's fast, handles real-world PDFs well, and can read from bytes directly — which is what I have when someone uploads a file, not a file path. The build plan recommended it as the default first choice.
Why do you chunk documents?	Two reasons. Embedding models can only take so much text per call. And even if they could take a whole book, the resulting vector would be a blurry average of everything in the book — retrieval needs each chunk to represent one idea, not a hundred.
Why overlap between chunks?	So sentences on chunk boundaries aren't lost. If an important phrase gets split in half between chunk 5 and chunk 6, neither one alone matches a query about it. Overlap means the end of chunk 5 shows up at the start of chunk 6, so at least one of them contains the whole thought.
Why did you start with a dumb chunker instead of a semantic one?	Because I can't tune what doesn't run yet. Building the whole pipeline dumb-first lets me verify end-to-end retrieval, then upgrade the chunker only if quality is poor. Starting with a semantic chunker means I spend days tuning it before knowing if anything downstream works.
What is a Qdrant point?	One stored record. It has three parts: an ID, the vector, and a payload — the payload is any JSON metadata I want, like the <code>file_id</code> , page number, and the original chunk text. I store the text in the payload because when I retrieve later, I need the actual words to send to the LLM.
Why deterministic UUIDs for point IDs?	So re-ingesting the same file doesn't create duplicates. UUID5 gives the same UUID for the same input, so chunk 3 of file X is always the same ID. Qdrant treats the second upsert as an update instead of a new point. Idempotent by design.

What happens if your embedding model changes?	If the new model outputs a different dimension, I'd have to recreate the Qdrant collection with the new size and re-ingest everything. That's why I pinned the model + dimension in <code>.env</code> up front and check the dimension in three places — startup, after each embedding, and against the live collection. If something's wrong I fail loudly instead of silently corrupting the DB.
Why separate services from API routes?	So the business logic doesn't depend on HTTP. My <code>ingest_pdf</code> function takes bytes and returns a report — I can call it from a test script, a CLI, or a background job just as easily as from the API. The route just handles the HTTP glue: read the upload, validate, call the service, return JSON.
Why 201 Created instead of 200 OK for the ingest endpoint?	Because 201 specifically means "you made a new resource" — which is what an ingest is. 200 is generic. Being precise matters when something like n8n branches on the response code.
What happens to a scanned PDF right now?	The parser returns empty text for its pages, the chunker returns no chunks, and my service reports 0 chunks created. It doesn't crash — it just processes nothing. OCR support comes in Phase 4, where I'll detect a page with too little text and run Tesseract on it.
How do you know your retrieval will find the right chunk later?	Because I verified semantic similarity works — "revenue in Q2" and "income last quarter" scored 0.72 similar, versus 0.45 against an unrelated sentence about weather. When a query embedding is close to a stored chunk's embedding, cosine similarity in Qdrant will find it, regardless of exact word overlap.
Walk me through what happens when someone POSTs a PDF to your endpoint.	The route validates the content-type and extension, reads the bytes, and checks the size limit. It calls <code>ingest_pdf</code> which calls <code>ensure_collection</code> , generates a <code>file_id</code> , then extracts text page by page with PyMuPDF. Each page's text is chunked with overlap. Every chunk gets embedded — the client POSTs each chunk to Ollama, gets 768 floats back, and enforces the size check. Then all chunks + vectors + metadata (<code>page_number</code> , <code>file_id</code> , <code>filename</code> , <code>text</code>) get upserted to Qdrant in one call with deterministic UUIDs. The route returns a 201 with a JSON report saying how many pages processed and chunks created.

Day 2 · Phase 1 complete. Ingestion pipeline working end to end with real semantic embeddings, page-number metadata, idempotent upserts, and a clean multi-layered code structure. Next: Phase 2 — semantic search endpoint that returns matching chunks with sources.